RESEARCH IMPLEMENTATION HANDOFF / 01

Smooth graph travel-time fields on SE(3)

Architecture-neutral implementation specification | 11 October 2026

Objective and user intent

Implement a learned smooth parametrization of a transformed pairwise travel-time graph. Given start pose x and goal pose y, predict smooth latent quantities Z(x,y), then recover a possibly nonsmooth travel time through an explicit decoder D. Use intrinsic Eikonal and Bellman constraints to train and evaluate the result. The network architecture is deliberately unspecified; implement the geometry, decoder, losses, data, planner, and tests behind a replaceable model interface.

```
p = (x, y)
H_theta(p) = (G(x), G(y), Z_theta(x,y))
T_theta(x,y) = D(H_theta(x,y))
dT = dD o dH_theta
```

What is fixed in this specification

Start with a fully actuated rigid body, no obstacles, constant positive translation and rotation speed scales, and an ellipsoidal combined speed limit. Use a fixed smooth pose embedding G and a two-scalar latent output Z=(a,b). The decoder is analytic. Test this known solution before attempting obstacle-aware learning. Use separate endpoint Eikonal residuals, rather than one joint-gradient residual.

What is established, and what remains a hypothesis

Established for this baseline: exact travel time has a smooth two-scalar latent representation; the decoder reconstructs rotational cut-locus corners; intrinsic derivatives and the chain rule give the correct Eikonal residual away from singularities.

Not established: easier neural optimization, convergence of sampled PDE training to the correct viscosity solution, a globally smooth target for arbitrary obstacle environments, or novelty relative to existing transformed distance fields and latent metric models. The graph description alone is not an additional constraint on the learned time.

Scope of the first deliverable

Deliver runnable code with a pluggable network, deterministic data generation, an analytic oracle, supervised and physics-informed training modes, derivative verification, pose-space rollout, and machine-readable metrics. Do not present obstacle-free learning as outperforming the analytic formula: the oracle requires no learning. Its purpose is to verify the proposed representation and optimization.

Assumptions

Poses lie in one connected free domain. The baseline has no acceleration limits, inertia, nonholonomic constraints, or joint limits. It plans a rigid-body pose, not a robot arm configuration. For an arm, pose feasibility and whole-arm collisions require additional modeling. All times are finite in the baseline.

Pose geometry and motion model

Use physical tangent directions; never differentiate redundant matrix entries as if they were independent motions.

Pose conventions

```
x = (t_x, R_x),  t_x in R^3,  R_x in SO(3)
x * y = (t_x + R_x t_y, R_x R_y)
G(x) = concat(t_x, vec(R_x))            # 12 entries
H_theta(x,y) in R^26                   # 12 + 12 + 2
```

The transformed graph has intrinsic dimension 12, not 26. Retaining a fixed embedding G makes H injective and full-rank on the pose-pair tangent space when Z is smooth. A two-scalar latent replaces the single time coordinate, so the ambient dimension differs from the earlier R^25 original graph. Do not train G in the first implementation.

Default cost and controls

```
||v||^2 / s_t^2 + ||omega||^2 / s_r^2 <= 1
s_t > 0 : translation speed scale (m/s)
s_r > 0 : angular speed scale (rad/s)
```

Equivalently use normalized controls u=(u_t,u_r), ||u||<=1, with body twist (v,omega)=(s_t u_t,s_r u_r). This specifies an identity norm in a scaled twist frame, not in Euler-angle or matrix coordinates. Default dimensionless experiment values: s_t=s_r=1; retain units and scale parameters in every API.

Scaled tangent derivatives

```
E_1..E_3: pure body translations scaled by s_t
E_4..E_6: pure body rotations scaled by s_r
c_x[i] = d/de T(x exp(e E_i), y) at e=0
c_y[i] = d/de T(x, y exp(e E_i)) at e=0
||c_x||^2 = 1,     ||c_y||^2 = 1
```

Implement derivatives by introducing zero local twist perturbations and differentiating through a numerically stable SE(3) exponential, or by equivalent analytic tangent Jacobians. Use series expansions around zero for Rodrigues and the SE(3) translation Jacobian; naive 0/0 branches can poison autodiff. Keep body/world conventions consistent.

Alternative models are separate configurations

Independent translation and rotation limits give a max decoder; an additive budget gives a sum decoder. Their Hamiltonians differ. Do not switch the decoder while retaining the ellipsoidal Eikonal loss. This handoff implements only the ellipsoidal model first.

Why not the original joint constraint?

For the chosen scaled metric, the true pairwise time satisfies both endpoint equations and therefore a joint squared norm of 2. A joint residual alone allows the two endpoint contributions to compensate incorrectly. The paper supplied by the user also constrains the two endpoints separately [1].

Exact latent target and decoder

The baseline gives a known smooth H and an exact nonsmooth time decoder.

Ground-truth construction

```
a_star = ||t_x - t_y||^2
b_star = (3 - trace(R_x^T R_y)) / 4
0 <= b_star <= 1
theta = 2 asin(sqrt(b_star))
phi(b) = 4 asin(sqrt(b))^2
Q = a / s_t^2 + phi(b) / s_r^2
D(a,b) = sqrt(Q)
```

Here a has units of squared length, b is dimensionless, and Q has units of squared time. The decoder ignores the G coordinates. The exact oracle is T_star=sqrt(a_star/s_t^2 + theta^2/s_r^2). It follows from the weighted product metric: translation follows a straight line and rotation follows a shortest angular geodesic, with compatible constant rates.

Oracle implementation must be independent

Compute oracle theta robustly using atan2: c=(trace(Q_rot)-1)/2 and s=||vee(Q_rot-Q_rot^T)||/2, then theta=atan2(s,c). Here vee extracts the axial vector of a skew matrix. Validate exact and near-half-turn cases; use a stable SO(3) logarithm for oracle paths. Avoid using the same decoder code to generate all test labels, which could hide shared bugs.

Replaceable network contract

```
predict_raw(x_features, y_features) -> [batch, 2]
predict_latent(x,y) -> (a_hat, b_hat)
decode(a_hat,b_hat) -> T_hat
value(x,y) -> decode(predict_latent(x,y))
```

Require deterministic evaluation and at least twice differentiable operations on regular samples for Eikonal training. Hidden widths, depth, activation family, attention, residual connections, and optimizer scheduling architecture are open. Prefer a fixed representation based on rotation matrices; quaternion data must yield identical predictions for q and -q.

One optional smooth output adapter

For an initial unconstrained network interface, average raw outputs under endpoint exchange to enforce symmetry. Map raw_a through softplus and raw_b through sigmoid. This keeps a>0 and 0<b<1 on regular samples, but cannot represent exact zero or one at finite logits. Consequently endpoint and half-turn accuracy is approximate; document this rather than calling the representation exact. Make the adapter replaceable for future boundary-exact designs.

Do not hard-code a_star and b_star into the learned prediction path in the experiment intended to test learning H. Keep a separate analytic baseline. Latent supervision is available for debugging, but physics-only training must not secretly use oracle labels beyond explicitly declared anchor samples.

Losses and training objective

Separate mathematical constraints from optional regularizers and numerical stabilizers.

Endpoint Eikonal loss

```
c_x = (J_x^L H_theta)^T grad(D)
c_y = (J_y^L H_theta)^T grad(D)
L_E = mean[(||c_x||^2-1)^2 + (||c_y||^2-1)^2]
```

J_x^L uses the six scaled twist directions from page 2. In practice differentiate the composite scalar T directly; explicit full H Jacobians are unnecessary. Training this loss requires create_graph=True or its framework equivalent. Do not detach c_x or c_y when computing L_E.

Boundary and calibration terms

```
L_goal = mean[T_theta(x,x)^2] / tau_ref^2
L_T = mean[(T_theta(x,y)-T_star(x,y))^2] / tau_ref^2
L_Z = mean[(a_hat-a_star)^2 / L_ref^4
          + (b_hat-b_star)^2]
```

Compute L_goal through Q=a/s_t^2+phi(b)/s_r^2 rather than differentiating sqrt at zero. L_ref is the fixed translation scale of the training domain; tau_ref=L_ref/s_t. Use L_T and L_Z in supervised diagnostics. In physics-informed experiments, identify any small anchor set explicitly and report its size. Exact goal values may need a boundary-exact adapter; an inference-only override is not a training guarantee.

Finite-step Bellman term

```
target(x,y) = min_u [h + T_target(x_next(u),y)]
x_next(u) = x exp(h * hat(s_t*u_t, s_r*u_r))
L_B = mean[(T_theta(x,y)-stopgrad(target))^2]
      / tau_ref^2
```

Sample admissible ||u||<=1; reject infeasible swept transitions when obstacles are enabled. Stop trajectories upon reaching the goal and charge only elapsed time to arrival, rather than always h. For baseline evaluation include the exact endpoint-reaching action when the goal is reachable in the step. Finite action sampling approximates the minimization; finite constant-twist rollouts approximate the full dynamic-programming operator. Refine horizon/action resolution rather than claiming an exact Bellman solver.

Metric regularization and identifiability

Positivity follows from the decoder and range adapter; exchange averaging supplies symmetry. Triangle inequality is not automatic. Optionally penalize max(0,T(x,y)-T(x,z)-T(z,y))^2. Scalar time and PDE losses do not uniquely identify two latent outputs a,b: many decompositions can produce the same time. Use L_Z to test the intended analytic representation; do not interpret physics-only outputs as translation/rotation components without evidence.

Initial normalized objective: L=L_E+L_B+10 L_goal+lambda_anchor L_T. Use lambda_anchor=1 when anchors are enabled; set it to zero for a labeled no-anchor ablation. Treat these as starting values, log every component, and tune only on validation data.

Singularities and stable differentiation

The decoder reconstructs corners; the derivative loss is not defined at every pose pair.

Two singular sets

Diagonal: x=y, where true distance has a norm-type cusp. **Rotational cut locus:** relative angle pi, where two shortest rotation directions tie. The latent target remains smooth there, but the decoder is not smoothly invertible. Handle these points with value/boundary/Bellman losses and evaluation, not a fabricated classical gradient label.

```
phi_prime(b) = 4 asin(sqrt(b)) / sqrt(b*(1-b))
lim_(b->0) phi_prime(b) = 4
phi(b) = 4*b + (4/3)*b^2 + (32/45)*b^3 + ...
```

Use the series for small b to avoid removable singularities at zero. The singularity at b=1 is genuine: do not remove it by claiming the derivative is finite. For forward decoding use 2*atan2(sqrt(b),sqrt(1-b)) if useful; it does not cure derivative singularity.

Regular-sample policy

Use float64 for geometry/derivative verification. Start with a diagonal exclusion T_star/tau_ref<1e-3 and a cut-locus band 1-b_star<1e-4 for L_E. These are initial numerical choices, not mathematical constants. Apply exclusions using known input geometry in the analytic baseline, never solely predicted b, which could let the model evade its loss. Sweep the bands and report their sizes and performance separately.

The range adapter should prevent invalid b on regular training inputs. Clamp only tiny floating-point domain excursions in analytic evaluation, track their frequency, and do not silently use hard clamping as the learned training parameterization. Exact singular samples remain in value tests, including b=1 and x=y.

Optional squared-time residual

```
Q_theta = T_theta^2
c_Qx[i] = derivative of Q_theta along scaled E_i
||c_Qx||^2 = 4*Q_theta       # regular region
(and likewise for y)
```

This avoids the outer sqrt derivative near the diagonal, but not the half-turn singularity in phi. It also admits the spurious solution Q=0 after clearing factors. Never replace the original residual with this equation without calibration, boundary/near-goal information, Bellman constraints, and a collapse test. Satisfying it is not by itself proof of a valid travel-time solution.

Autodiff checks

Compare analytic directional derivatives, automatic differentiation, and central finite differences away from both singular sets. Sweep finite-difference step sizes; check convergence before roundoff. Verify mixed parameter/input derivatives remain finite during a full optimizer step. On singular sets check values and one-sided limits instead of demanding matching two-sided derivatives.

Data and experiment sequence

Do not skip the analytic stage; the baseline is small enough to diagnose every component.

Sampling specification

Sample translations independently in [-1,1]^3 meters by default. Sample Haar-uniform rotations via normalized independent four-dimensional Gaussian quaternions, converted to matrices. In a regular training batch use 60% uniform pairs, 20% small relative pose perturbations, and 20% near-half-turn pairs. Draw a separate diagonal batch. Near pi, sample random axes and logarithmically distributed angular offsets, with exact pi pairs reserved for value tests. Report the induced distribution rather than calling this mixture uniform.

Use random seeds 0, 1, and 2. Keep fixed validation/test pairs separate from the training stream; start with 10,000 pairs in each held-out set plus dedicated singularity suites. Training size, parameter count, and compute should be matched across architectures. Save sampler states for restart reproducibility.

Stage A / analytic geometry, no neural training

Implement pose utilities, latent oracle, decoder, tangent derivatives, and the exact optimal path. Verify decoder times against the independent angle oracle and verify both endpoint Eikonal equations on regular pairs. This isolates geometry and decoder errors before optimization.

Stage B / representation diagnostic

Plug in a user-chosen smooth network. Train with L_Z and L_T, plus diagonal value constraints. Verify it can reproduce the analytic latent targets and decoded times. This is supervised validation of learnability, not the physics-informed contribution.

Stage C / physics-informed learning

Train with endpoint Eikonal, Bellman, and goal losses. Run both a declared small-anchor version (start with 1,024 fixed regular pairs) and a no-anchor ablation. Optionally warm-start from Stage B, but label that experiment separately from training from scratch. Use an EMA target network for Bellman labels (initial decay 0.995); target predictions are detached.

Suggested starting optimizer: Adam, learning rate 1e-3, batch size 256, global gradient clipping at 1, 20,000 updates, validation every 500 updates. Reduce batch size for second-derivative memory limits. These are provisional defaults. Select checkpoints by held-out value and rollout metrics, not training Eikonal residual alone. Record wall-clock time and peak memory.

Stage D / fair comparisons

Compare analytic oracle, direct scalar T network, smooth graph+decoder network, and an implemented or available latent-metric baseline inspired by [1]. Hold data, optimization budget, pose representation, speed model, and planner fixed. Ablate latent supervision, Bellman loss, boundary treatment, and singular-band size. Report uncertainty across seeds.

Stage E / obstacle extension, separate research branch

Only after A-D pass, add a collision-free pose domain and either hard feasibility or a declared position-dependent speed. The two-output decoder no longer implies smooth latent targets or separate physical meanings for a and b. Consider richer latents or branch decoders, but treat them as new hypotheses requiring tests; do not assert a universal smoothing theorem.

Path extraction and evaluation

Use actual SE(3) updates and assess a trajectory, not just a PDE residual.

Exact oracle path

```
t(lambda) = (1-lambda)*t_x + lambda*t_y
R(lambda) = R_x exp(lambda * log(R_x^T R_y))
0 <= lambda <= 1
```

This is a shortest path for the chosen obstacle-free product metric. At a half-turn choose a consistent valid axis; both tied directions can be optimal. Parameterize lambda by elapsed time/T_star for unit normalized speed. This is a test oracle, not the learned planner.

Gradient rollout on regular pairs

```
u = -c_x / ||c_x||
xi = concat(s_t*u[:3], s_r*u[3:])
x_next = x exp(h * hat(xi))
```

Choose a step size small enough to avoid goal overshoot and excessive local integration error. Start h=0.02*tau_ref; cap translation and rotation increments, and backtrack if decoded value fails to decrease. Stop when both translation and angular goal errors fall below tolerance. If gradient is near zero, nonfinite, or cut-locus sensitive, switch to the sampling planner and log the event. Never differentiate only the transformed latent height as a substitute for decoded time.

Sampling rollout

Use an initial budget of 128 normalized twist candidates per step, including the zero action and previous action; choose the candidate minimizing elapsed cost plus T_theta(next,y). A receding horizon can be added without changing the value API. For fair experiments use identical action budgets across learned fields. Check all transitions against the same collision model if obstacles are introduced; endpoint-only checks do not prove swept collision freedom.

Metrics to record

Field: absolute/RMSE time error, relative error away from tiny times, both endpoint PDE residuals, symmetry/goal/triangle errors, latent errors only when identifiable, and nonfinite evaluations.

Singular neighborhoods: stratify by relative angle (0-90, 90-150, 150-170, 170-179, 179-180 degrees), by goal distance, and by excluded Eikonal bands. Report exact-half-turn value errors separately. Plot slices crossing the cut locus: true T has a corner while analytic b is smooth.

Planning: success rate, terminal translation/angular errors, cost ratio to oracle, integration steps, runtime, fallback frequency, and collisions for the later obstacle branch. Compute path cost as the integral of the specified weighted twist norm, not an unweighted norm of pose coordinates.

Initial numerical gates

In float64, target oracle-decoder value agreement within 1e-8 times an appropriate time scale on ordinary pairs and Eikonal squared-norm error below 1e-6 away from singular bands. Use looser, separately justified tolerances extremely close to pi. Learned targets: report achieved values; provisional goals are median relative error below 1%, 95th percentile below 5%, and at least 99% success on the simple obstacle-free benchmark. These are targets, not promised results.

Code implementation contract

The receiving coding agent should implement the following interfaces before selecting a network.

```
geometry/se3.py       compose, inverse, exp, log, hat
geometry/tangents.py  scaled endpoint derivatives
fields/oracle.py      analytic latent, time, path
fields/decoder.py     domain-safe forward + series
models/interface.py  replaceable pair-to-latent module
losses/eikonal.py     separate endpoint residuals
losses/bellman.py     admissible finite-step targets
data/poses.py         samplers + fixed held-out suites
planning/rollout.py   gradient and sampling planners
train.py             staged experiments + checkpoints
evaluate.py          fields, singularities, trajectories
tests/               geometry, decoder, gradients, loss
configs/             all scales, masks, seeds, budgets
```

Training-step pseudocode

```
x, y = sample_training_pairs()
a, b = model.predict_latent(x, y)
T = decoder(a, b)
regular = regular_mask_from_input_geometry(x, y)
cx, cy = tangent_gradients(model.value, x, y,
                           create_graph=True)
LE = endpoint_residual(cx[regular], cy[regular])
LB = bellman_loss(model, target_model, x, y)
Lgoal = diagonal_squared_time_loss(model)
L = LE + LB + 10*Lgoal + declared_anchor_loss
optimizer.zero_grad(); L.backward(); clip_grad_norm()
optimizer.step(); update_target_model()
```

Implementation correction to the schematic: construct derivative-dependent decoder evaluations only on regular samples. Merely masking a nonfinite derivative after computing it can still propagate NaNs. Use separate forward batches for singular value supervision and regular PDE differentiation.

Mandatory tests

Identity/composition/inverse and exp/log round trips; rotation orthogonality and determinant; quaternion-sign invariance; endpoint-swap symmetry; common rigid-transform invariance of the analytic oracle; pure translation and pure rotation times; exact and near-pi cases; diagonal values; stable first and parameter/input derivatives; both endpoint residuals; reproducible sampling; no accidental oracle-label leakage; full rollout to the goal.

Checkpoint and report requirements

Save architecture metadata (when chosen), state dictionaries, optimizer and target states, scales, sampler/RNG states, code revision, dtype, exclusions, loss weights, and training mode. Export a JSON/CSV evaluation report and representative path/field plots. The README must state what was actually implemented and distinguish proven identities from empirical claims.

Instruction to the next ChatGPT instance: inspect the existing repository first, preserve user changes, implement Stage A before a neural model, and expose the architecture as an interchangeable component. Do not silently claim the obstacle case is solved or reinterpret the cost model. Discuss material deviations from this specification.

Theory boundaries and related work

Use this page to frame claims accurately when implementing, evaluating, or writing a paper.

The claim this implementation can test

A smooth latent representation with an explicit nonsmooth decoder may approximate the pairwise travel-time field more accurately or train more reliably near rotational cut loci than a directly smooth scalar network. This must be measured under matched data, models, and planning budgets. The baseline itself has a known analytic solution; successful fitting validates the mechanism, not a need for learning.

Claims this specification does not support

No proof that an arbitrary learned H is an embedding unless the fixed G coordinates are retained. No universal smooth finite-dimensional latent representation with a prescribed stable decoder for all environments. No guarantee that low sampled Eikonal residual selects the viscosity solution. No completeness, collision-safety, or optimality guarantee for a finite-budget learned rollout. No automatic triangle inequality from the graph parameterization.

Sources and relation to the proposal

[1] Ni, Pan, Qureshi. Physics-Informed Temporal Difference Metric Learning for Robot Motion Planning. ICLR 2025. Primary baseline supplied by the user. Separate endpoint Eikonal equations, finite-step TD training, a nonsmooth latent metric, and sampling-based MPC. The proposed Jacobian expression is a chain-rule reformulation, not a new PDE.
https://arxiv.org/abs/2505.05691

[2] Fainstein, Siless, Iarussi. DUDF: Differentiable Unsigned Distance Fields with Hyperbolic Scaling. CVPR 2024. Closely related transformed-target idea for unsigned surface distance, with a modified Eikonal formulation. It does not establish a general SE(3) obstacle-aware graph decoder.
https://arxiv.org/abs/2402.08876

[3] Li, Qiu, Calinon. A Riemannian Take on Distance Fields and Geodesic Flows in Robotics. Preprint 2024. Geometry-aware neural Eikonal learning and motion generation; relevant to intrinsic derivatives and metric specification.
https://arxiv.org/abs/2412.05197

[4] Darbon, Dower, Meng. Neural network architectures using min-plus algebra for solving certain high dimensional optimal control problems and Hamilton-Jacobi PDEs. Preprint 2021; revised 2023. Prior work for minimum-based value-function representations. Relevant if the obstacle extension adopts smooth branches and a min decoder.
https://arxiv.org/abs/2105.03336

Provenance

This document formalizes the user's proposed transformed-graph approach and the subsequent discussion. Exact SE(3) baseline formulas are mathematical derivations under the stated product metric. Hyperparameters and training stages are implementation recommendations, not findings from completed experiments. No network architecture has been selected and no model has been trained as part of this handoff.